

Verifying Distributed Algorithms with Interactive Theorem Provers: A Survey of Approaches

MAXWELL MOIR, University of Canterbury, New Zealand

Formal verification is a topical field in modern Computer Science. Proving the correctness of programs is not only elegant in a theoretical sense, but is often necessary in mission-critical applications. Due to the difficulty in formalizing programs, reliability in industry is often left to automated testing. However, in specific circumstances where correctness must be ensured, formal proofs may be required. This survey reviews the different approaches to verifying the correctness of distributed algorithms.

CCS Concepts: • **Theory of computation** → **Logic and verification**.

Additional Key Words and Phrases: distributed algorithms

ACM Reference Format:

Maxwell Moir. 2026. Verifying Distributed Algorithms with Interactive Theorem Provers: A Survey of Approaches. *J. ACM* 37, 4, Article 111 (August 2026), 3 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 INTRODUCTION

With increasing societal dependance on software, ensuring the effective operation of the tools we use has become more important.

One method for reasoning about the correctness of programs is formal verification. Distributed systems underpin many of the applications that we use every day. Distributed systems are notoriously difficult to verify due to ...

This survey aims to explore attempts to formally verify distributed algorithms, both theoretically and in practice. It provides a taxonomy of different theorem-based verification approaches and frameworks, reviews several common tools, and contrasts with other verification approaches.

Other approaches to the verification of distributed algorithms are outside the scope of this survey, except when used as comparison.

2 PRELIMINARY

2.1 Distributed Protocols

Distributed computing is ...

2.2 Interactive Theorem Provers

2.3 Verification

3 APPROACHES

Nam id fermentum dui. Suspendisse sagittis tortor a nulla mollis, in pulvinar ex pretium. Sed interdum orci quis metus euismod, et sagittis enim maximus. Vestibulum gravida massa ut felis

Author's address: Maxwell Moir, University of Canterbury, Christchurch, New Zealand, mmo199@uclive.ac.nz.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 0004-5411/2026/8-ART111

<https://doi.org/XXXXXXX.XXXXXXX>

suscipit congue. Quisque mattis elit a risus ultrices commodo venenatis eget dui. Etiam sagittis eleifend elementum.

There are several common frameworks used in the verification of distributed protocols. Each of which has its own benefits and detractions.

Common Frameworks: Verdi [5] Iron Fleet [2] Diesel [4] Veil [3] TLAPS [1]

Common Languages: Coq Lean Isabelle Dafny

Common Problems Paxos Raft

Verified system examples

Byzantine fault tolerance

Compositional and modular verification

Proof techniques: Invariant refinement

Adjacents

4 APPLICATIONS

Nam id fermentum dui. Suspendisse sagittis tortor a nulla , in pulvinar ex pretium. Sed interdum orci quis metus euismod, et sagittis

4.1 Verified protocols

Nam id fermentum dui. Suspendisse sagittis tortor a nulla , in pulvinar ex pretium. Sed interdum orci quis metus euismod, et sagittis enim maximus. Vestibulum gravida massa ut felis suscipit congue. Quisque mattis elit a risus ultrices commodo venenatis eget dui. Etiam sagittis eleifend elementum.

4.2 Verified systems

Nam id fermentum dui. Suspendisse sagittis tortor a nulla , in pulvinar ex pretium. Sed interdum orci quis metus euismod, et sagittis enim maximus. Vestibulum gravida massa ut felis suscipit congue. Quisque mattis elit a risus ultrices commodo venenatis eget dui. Etiam sagittis eleifend elementum.

r plants. Non-safety-critical applications include a web browser and few social applications: a conferencing system, a social media platform. In spite of the potential of today's verification technology and its successes as documented here, formal verification is still not mainstream. The key obstacle is the high entry level: each project needs one or more verification experts. Companies may feel that it will be easier to find many testers than a few verification-savvy developers. Another concern is the rate of change of modern software systems. It is now common to practice "continuous development", producing new versions once or more daily. In principle every such version requires re-verification. The good news, reported by several projects (seL4, Cocon, Amazon s2n, iFACTS, and Dutch tunnel) is that re-verification costs much less than the original verification. seL4 reports that verifying a new and large feature (a new data structure for API calls) cost less than 10An alternative to re-verification is reuse of verified elements. Reviewing the projects revealed a number of cases of successful reuse, for example in OS projects (mCertiKos 6.5.10, Hyper-V, and PikeOS). Particularly interesting is the reuse by VeriCert of

5 ALTERNATIVE VERIFICATION APPROACHES

Model checking,

6 FUTURE DIRECTIONS

Nam id fermentum dui. Suspendisse sagittis tortor a nulla , in pulvinar ex pretium. Sed interdum orci quis metus euismod, et sagittis enim maximus. Vestibulum gravida massa ut felis suscipit

congue. Quisque mattis elit a risus ultrices commodo venenatis eget dui. Etiam sagittis eleifend elementum.

7 CONCLUSION

Nam id fermentum dui. sagittis tortor a nulla mollis, in pulvinar ex pretium. Sed interdum orci quis metus euismod, et sagittis enim maximus. Vestibulum gravida massa ut felis suscipit congue. Quisque mattis elit a risus ultrices commodo venenatis eget dui. Etiam sagittis eleifend elementum. Test 01

8 METHODOLOGY

8.1 Literature Search Strategy

To identify relevant papers, we used a systematic literature search strategy. We chose to query two databases, the ACM digital library and IEEE Xplore. These databases were accessed on April 17 2026.

To ensure the quality of of this review, we decided to follow the PRISMA framework to inform our search strategy. The search strategy can be summarized into multiple steps. First, we curated relevant keywords and constructed search queries to compile relevant papers based on the guiding questions of the survey. This led to the following search query:

("proof assistant" OR "theorem prover" OR "mechanized verification" OR "formal verification") AND ("distributed protocol" OR "distributed algorithm")

This query yielded 213 results from the ACM Digital Library and 49 results from IEEE Xplore.

8.2 Inclusion and Exclusion Criteria

These results were then screened based on standard inclusion and exclusion requirements. This reduced the result count to 37 for ACM and XX for IEEE Xplore. From this point, the results were expanded by adding commonly cited papers.

REFERENCES

- [1] Denis Cousineau, Damien Doligez, Leslie Lamport, Stephan Merz, Daniel Ricketts, and Hernan Vanzetto. 2012. TLA+ Proofs. *Proceedings of the 18th International Symposium on Formal Methods (FM 2012)*, Dimitra Giannakopoulou and Dominique Mery, editors. Springer-Verlag Lecture Notes in Computer Science 7436 (Jan. 2012), 147–154.
- [2] Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jacob R. Lorch, Bryan Parno, Michael L. Roberts, Srinath Setty, and Brian Zill. 2015. IronFleet: Proving Practical Distributed Systems Correct. In *Proceedings of the 25th Symposium on Operating Systems Principles (SOSP '15)*. Association for Computing Machinery, New York, NY, USA, 1–17. <https://doi.org/10.1145/2815400.2815428>
- [3] George Pirlea, Vladimir Gladshtein, Elad Kinsbruner, Qiuyan Zhao, and Ilya Sergey. 2025. Veil: A Framework for Automated and Interactive Verification of Transition Systems. In *Computer Aided Verification: 37th International Conference, CAV 2025, Zagreb, Croatia, July 23-25, 2025, Proceedings, Part III*. Springer-Verlag, Berlin, Heidelberg, 26–41. https://doi.org/10.1007/978-3-031-98682-6_2
- [4] Ilya Sergey, James R. Wilcox, and Zachary Tatlock. 2017. Programming and Proving with Distributed Protocols. *Proc. ACM Program. Lang.* 2, POPL (Dec. 2017), 28:1–28:30. <https://doi.org/10.1145/3158116>
- [5] James R. Wilcox, Doug Woos, Pavel Panchekha, Zachary Tatlock, Xi Wang, Michael D. Ernst, and Thomas Anderson. 2015. Verdi: A Framework for Implementing and Formally Verifying Distributed Systems. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '15)*. Association for Computing Machinery, New York, NY, USA, 357–368. <https://doi.org/10.1145/2737924.2737958>